GRIDGUARD KHARTOUM

PPO-Assisted Load Shedding and Energy Management for Developing Countries

Graduation Project Final Report

Murtada Asim Ibrahim Bakhit (18-317)
Suliman Elawad Suliman Babiker (18-310)
Abdalhay Abass Mobarak Abass (15-306)

Offline research simulation | Audited synthetic evidence | GPU-first runtime

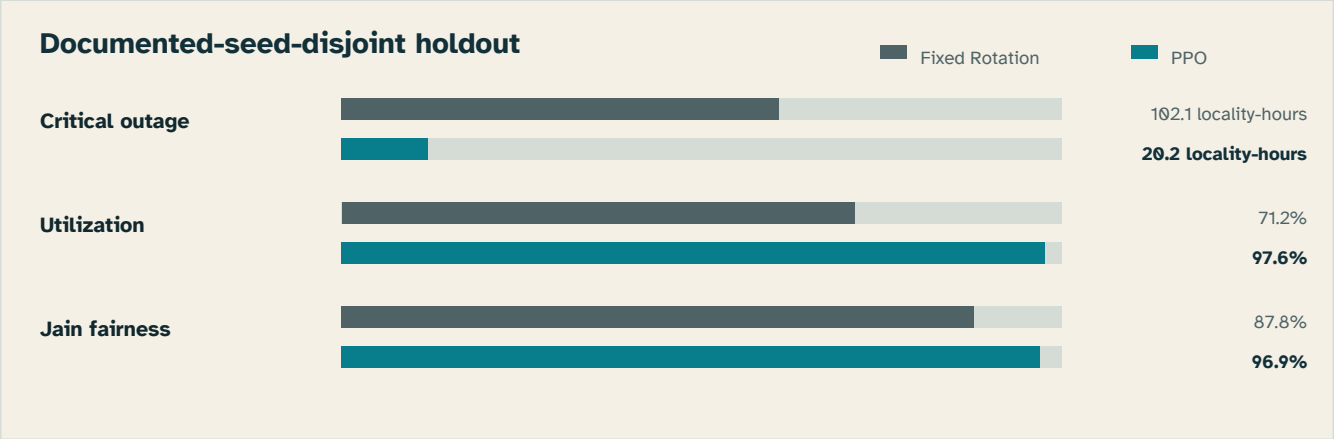
August 2026

Abstract

GridGuard Khartoum is an offline, GPU-first research simulation for proposal-aligned load shedding across seven Khartoum State localities. It compares a frozen PPO controller with Fixed Rotation on identical synthetic 24-hour shortage scenarios. The release separates an accepted reproduction whose seeds participated in model selection from a documented-seed-disjoint holdout on seeds 81001-81010: 320 independent base draws, 960 matched scenario conditions, and 1,920 controller evaluations. Critical-outage locality-hours, fairness, and utilization are the three primary outcomes; the accepted reward is a Training-objective score (diagnostic), and power balance is reported separately as safety. A strict `data/model/training-provenance.yaml` exposes the historical network, behavior-cloning initialization, optimizer settings, seeds, selection, source hashes, and evidence category of each recovered field. The selected seed-43 run used 600 PPO iterations, 153,600 episodes, and 3,686,400 transitions; the three-seed candidate experiment totaled 1,800 iterations, 460,800 episodes, and 11,059,200 transitions. Hardware and convergence are explicitly selected-run facts, and historical promotion gates are not independent headline evidence. The robustness supplement also covers 25 parameterizations and compares the display forecast with one-hour persistence. All traces are synthetic, the operating domain is shortage-only, and the work is decision-support research rather than a deployed controller or field-performance claim.

Documented-seed-disjoint holdout: 320 independent base draws x 3 shortage levels = 960 matched scenario conditions; 1,920 controller-scenario evaluations. Seeds are disjoint only from every documented training, validation, and model-selection seed in data/model/training-provenance.yaml. The holdout was not preregistered or independently custodied and cannot rule out undocumented activity.

Sensitivity intervals are descriptive, pointwise 95%, and unadjusted for multiple comparisons; they do not support family-wide significance claims.



Outcome	Fixed Rotation	PPO	Paired change
Critical outage	102.1 locality-hours	20.2 locality-hours	-81.9 locality-hours
Jain fairness	87.76%	96.91%	+9.15 pp
Utilization	71.21%	97.61%	+26.39 pp

Comparator boundary: Fixed Rotation is supply-blind, limited to Off and Full Service, and cannot select Critical Only. Synthetic decision support only, not field evidence. No behavior-cloning-only ablation was run, so this comparison cannot isolate the incremental effect of PPO fine-tuning.

Contents

Abstract 2
System at a glance 4
Introduction 5
Related Work 6
- Sudan energy-access context . . . 6
- Long-duration shortage, rationing, and fairness . . . 6
- Critical loads and operational constraints . . . 7
- Reinforcement learning in energy control . . . 7
- Demand forecasting and evaluation . . . 7
- Position of GridGuard Khartoum . . . 8

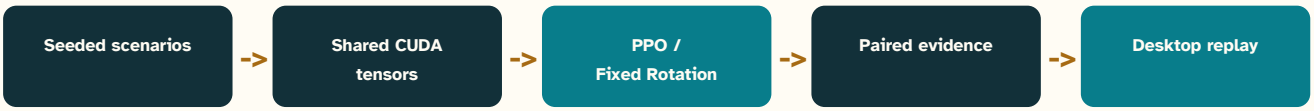
Background 9
Methodology 10
- Study objective and claim boundary . . . 10
- Simulation domain . . . 10
- Controller decisions and dispatch . . . 12
- Reward . . . 13
- Outcome definitions and signs . . . 13
- Accepted reproduction design . . . 14
- Paired uncertainty calculation . . . 15
- Documented-seed-disjoint robustness, sensitivity, and forecast design . . . 16
- Causal display-only demand projection . . . 16
- Reproducibility and provenance . . . 17
- Limitations and threats to validity . . . 18

Implementation 19
- System overview . . . 19
- Module map . . . 20
- Domain and tensor contracts . . . 21
- GPU-first execution . . . 21
- PPO artifact and inference . . . 21
- Scenario, dispatch, and metrics pipeline . . . 22
- Accepted reproduction evaluation and statistics . . . 23
- Documented-seed-disjoint robustness pipeline . . . 24
- Evidence provenance and fail-closed behavior . . . 24
- Display-only forecast . . . 25
- Offline administrative map . . . 25
- Desktop presentation . . . 26
- Runtime resources and packaging . . . 27
- Verification strategy . . . 27
- Deliberate implementation limits . . . 28

Desktop presentation 29
Results 31
- Result statement . . . 31
- Results provenance and evidence roles . . . 31
- Documented-seed-disjoint primary outcomes . . . 32
- Accepted reproduction context . . . 32
- Sensitivity across illustrative demand allocation . . . 33
- Synthetic forecast comparison . . . 33
- Baseline interpretation . . . 34
- Interpretation against the proposal questions . . . 34
- Limitations of the result . . . 34

Discussion 36
Conclusion 37
References 38

System at a glance



One severity-neutral base-draw batch is reused across all three shortage levels, and the same resulting scenario tensor is passed to both controllers. All performance-critical simulation, policy inference, metric, and bootstrap operations remain on cuda:0; final display and canonical evidence payloads cross explicitly to host memory. Startup verifies the NVIDIA runtime, evidence hashes and semantics, PPO artifact, and a CUDA smoke inference before the first presentation window appears.

Claim boundary: every numeric result in this report describes the locked synthetic simulator and protocol. It is not operational evidence from Sudan's electrical grid.

Introduction

Planned electricity shortages create an allocation problem: limited supply must be distributed while essential services remain protected and repeated exclusion of the same communities is avoided. The accepted proposal asks whether artificial intelligence can support that decision in a simplified model of Khartoum State. GridGuard Khartoum turns that question into a reproducible software experiment and an offline presentation tool. Its controller comparison is deliberately narrow: one frozen PPO policy is evaluated against one transparent but supply-blind Fixed Rotation schedule on identical synthetic scenarios.

The project has two deliverables that must not be confused. The scientific package produces and verifies matched controller evidence. The desktop application explains that evidence and can run a separate live scenario for demonstration. Live values never replace the verified documented-seed-disjoint robustness supplement used by the Results page. The system also shows real locality boundaries and the White Nile, Blue Nile, and Nile for geographic orientation, but those layers contribute no demand, topology, controller input, or metric value.

Related Work

GridGuard Khartoum sits at the intersection of electricity rationing, fair allocation, critical-load protection, reinforcement learning, and short-horizon demand forecasting. These fields contain substantially richer physical and institutional models than the present application. This review therefore positions the project as a bounded, offline decision-support prototype and identifies the evidence that would be required to move beyond its Khartoum-inspired synthetic case. It does not treat geographic labels or an administrative map as validation of an electrical model.

Sudan energy-access context

The World Bank's 2025 appraisal-stage project document describes severe disruption to Sudan's electricity sector, including grid damage, generation shortfalls, fuel-supply constraints, and outages affecting households and essential services [9]. It also describes a current emphasis on decentralized energy access for public facilities and other underserved uses. This establishes the importance of resilient access and critical services in the national context. It does not provide the feeder measurements, hourly locality demand, switching constraints, or operating records needed to parameterize this simulator.

Accordingly, reference [9] is used only for problem context. It is not used to infer the project's 700 MW peak, locality demand weights, sector shares, hourly load profiles, shortage fractions, or service priorities. Those quantities are proposal-constrained or illustrative assumptions. The scientific path in this release is therefore not calibrated to the Khartoum grid. Calibration would require time-aligned utility measurements, network and feeder topology, generation and import availability, equipment limits, outage and restoration histories, critical-facility inventories, operator rules, weather and calendar effects, and a defensible mapping from physical assets to the seven modeled localities.

Long-duration shortage, rationing, and fairness

Heggie et al. study rationing policy under persistent shortage, where demand routinely exceeds supply and a policy must balance fair and efficient allocation amid operational uncertainty [10]. That framing is closer to GridGuard's shortage episodes than emergency-only under-frequency load shedding, but their work also highlights why a rotation schedule is a policy choice rather than a neutral benchmark. In GridGuard, Fixed Rotation does not observe available supply, requests Full Service for three localities, and turns the other four Off. Its low utilization and critical-service performance must therefore be interpreted partly as consequences of that rule design.

Oluwasuji et al. formulate fair electric load shedding for developing-country settings as a resource allocation problem with heterogeneous household needs [11]. Their analysis motivates looking beyond aggregate energy served and treating fairness as an explicit outcome. GridGuard uses Jain fairness over cumulative locality service ratios, but this is a coarser abstraction: a locality is not a household, individual preferences are absent, and the fixed sector shares do not establish actual vulnerability. Sensitivity analysis can show whether conclusions change under specified perturbations; it cannot replace locally observed demand or community-defined service priorities.

These studies also limit how the comparison should be worded. Matched scenarios isolate controller differences under one generator, but they do not cancel misspecification shared by every scenario. The comparison can support statements about the implemented synthetic experiment. It does not establish that one policy would be fairer or more efficient in field operation.

Critical loads and operational constraints

Poudel and Dubey formulate critical-load restoration using distributed energy resources, network connectivity, radial-operation constraints, restoration timing, and resource availability [12]. Their restoration problem differs from GridGuard's daily shortage allocation, yet it demonstrates that protecting critical services normally depends on physical topology and operational limits as well as a priority label. GridGuard instead chooses one of three service tiers for each locality and then applies a single proportional scale factor when requested power exceeds supply. Within a selected locality, critical and non-critical requested load are reduced by the same factor.

The present critical-service metric is therefore evidence about the simulator's tier requests and proportional dispatch, not about feeder-level hospital restoration. A realistic extension would model feeder connectivity, switching feasibility, minimum service blocks, ramping and restoration rules, critical-facility demand, and explicit priority constraints. A mathematical dispatcher or episode-level optimum would also be needed before describing any comparator as an upper bound.

Reinforcement learning in energy control

Vázquez-Canteli and Nagy review reinforcement learning for demand response across buildings, storage, electric vehicles, appliances, and related flexible-demand applications [13]. Their review notes the importance of human comfort, feedback, non-stationarity, and coordination. Glavic surveys reinforcement learning across normal, preventive, emergency, and restorative electric-power control contexts and across several control levels [14]. Together, these reviews support PPO as a recognizable research approach while emphasizing that deployment depends on a faithful environment, constraints, and evaluation beyond a training objective.

GridGuard uses a frozen PPO actor trained in a synthetic simulator and performs deterministic offline inference. It does not learn online, command utility equipment, estimate network state, or enforce alternating-current power-flow constraints. The joint action contains 3^7 = 2,187 possibilities for seven localities; this direct enumeration grows as 3^N. Factored, autoregressive, or shared-locality policies with explicit constraint handling are possible future directions, not capabilities of this release.

The distinction between training and evaluation is also important. The episode reward is the training-objective score and shares components with several reported measures. It is therefore a diagnostic of objective attainment rather than independent outcome evidence. The proposal-aligned primary outcomes are critical-outage locality-hours, Jain fairness, and utilization, each reported separately from the training objective.

Demand forecasting and evaluation

Nti et al. review electricity load-forecasting methods, input variables, horizons, and error measures [15]. Their synthesis reports common use of historical consumption and contextual inputs and identifies MAE and RMSE among established error measures. This supports evaluating GridGuard's causal three-hour trailing-mean projection against a simple one-hour persistence baseline on common target hours.

That experiment is deliberately narrow. Both forecast methods operate on synthetic demand traces, so their MAE and RMSE measure consistency with this generator rather than accuracy for Khartoum. No result from it supports operational scheduling, general forecast superiority, or a real-world accuracy claim. A field study would require held-out metered data, clearly defined forecast horizons, seasonal and event coverage, missing-data rules, and comparisons with appropriate statistical and operational baselines.

Position of GridGuard Khartoum

GridGuard integrates established ideas into an offline Windows demonstration: a deterministic synthetic scenario generator, Fixed Rotation and PPO controllers, matched evaluation, explicit fairness and critical-service measures, hash-verified evidence, and an administrative locality display. The project does not make a claim of algorithmic novelty, grid deployment, or utility calibration. Its contribution is a traceable academic implementation that makes assumptions and evidence boundaries inspectable.

Within those boundaries, the review literature motivates three improvements now incorporated into the evaluation design: independent physical and service outcomes rather than reward alone, one-factor sensitivity tests for illustrative demand parameters, and a forecast comparison with standard error measures. It also identifies work that remains outside the approved architecture: a supply-aware comparator, feeder-constrained optimization, an episode-level optimal benchmark, real demand calibration, larger or factored action spaces, and recovery or surplus operation. Consequently, favorable synthetic results may justify further investigation; they do not establish safe or effective operation on Sudan's power system.

Background

Fixed Rotation is a useful academic baseline because its schedule is deterministic and easy to audit, yet it does not observe current shortage conditions, critical demand, or accumulated service fairness. PPO represents the contrasting learned strategy: it maps a causal observation of current supply, demand, previous tiers, and service history to one joint action across seven localities. The study therefore asks whether the learned mapping improves the proposal outcomes inside a controlled simulator, not whether it can operate Sudan's electricity network.

The simulation models 24 one-hour steps, five demand sectors per locality, and three service tiers. Hospital, water, and bakery demand is treated as critical. Mild, Moderate, and Severe shortages are deterministic transformations of the same severity-neutral stochastic base draws. This matched construction isolates controller differences and motivates the paired analysis. Administrative geometry comes from the pinned OCHA/HDX Sudan COD-AB revision v03 [7], while river orientation comes from pinned Natural Earth 1:10m data [8]. Both are packaged offline and verified independently of scientific evidence.

Methodology

Study objective and claim boundary

GridGuard Khartoum evaluates whether a trained PPO controller can allocate limited electricity more effectively than a traditional Fixed Rotation schedule in a simplified, software-only model of Khartoum State. The accepted proposal [1] defines this as a decision-support simulation, not a controller for a real electricity network. Accordingly, the evidence supports a comparison inside the stated synthetic scenario design only.

The controller comparison is PPO versus Fixed Rotation. Evidence has two roles. Seeds `50001-50003` reproduce the accepted historical comparison but participated in checkpoint promotion and model selection, so that artifact is labeled accepted reproduction, not an independently custodied final test. Seeds `81001-81010` form the bounded documented-seed-disjoint holdout: they are disjoint from every documented training, validation, and model-selection seed in `data/model/training-provenance.yaml`. The holdout was not preregistered or independently custodied and cannot rule out undocumented activity. The proposal permits prediction, rule-based logic, optimization, and reinforcement learning where appropriate; it does not require every method to be a controller in the final comparison. The presentation includes a small causal demand projection as a separate explanatory feature. It is not an input to PPO and does not change dispatch.

Simulation domain

The model uses seven Khartoum State localities in a fixed tensor order: Khartoum, Omdurman, Bahri, East Nile, Jebel Awlia, Karari, and Ombada. Each locality contains five demand sectors: Hospital, Water, Bakery, Residential, and Commercial. Hospital, Water, and Bakery are the three critical sectors.

Each episode contains 24 one-hour steps. Demand and supply are expressed as MW at each step; energy totals are the sum over the one-hour steps and are therefore reported in MWh. The simulator uses one global supply pool. It does not model feeders, voltages, frequencies, transmission constraints, or power flow.

The presentation locates those seven units using second-level administrative boundaries from OCHA/HDX Sudan COD-AB revision v03 [7], with White Nile, Blue Nile, and Nile linework from Natural Earth 1:10m rivers [8]. These verified layers are presentation context only. They are not electrical-network topology and do not enter scenario generation, controller observations, dispatch, reward, metrics, or evidence.

Synthetic demand

The system peak parameter is 700 MW. Locality peak shares are proportional to the locked weights below. Locked sector coefficients define the base peak allocation before distinct hourly profiles are applied, so instantaneous sector proportions can vary by hour.

Locality	Relative weight
Khartoum	1.00
Omdurman	0.95
Bahri	0.85
East Nile	0.80
Jebel Awlia	0.70
Karari	0.75
Ombada	0.90

Sector	Base peak coefficient	Profile assumption
Hospital	0.08	Nearly constant through the day
Water	0.07	Morning and evening peaks
Bakery	0.05	Early-morning and afternoon peaks
Residential	0.55	Morning and stronger evening peaks
Commercial	0.25	Midday peak

The sector curves are deterministic normalized profiles built from constant and Gaussian components. Each generated episode then receives independent uniform demand perturbations in the interval [-5%, +5%]. The perturbations are produced by a seeded PyTorch generator on the selected device.

Synthetic supply and shortage levels

Available supply is current total demand multiplied by a shortage fraction and an independent uniform perturbation in [-5%, +5%]. It is capped at 95% of demand so every episode remains a shortage case.

Shortage level	Base supply fraction	Additional shortage window
Mild	0.80	None
Moderate	0.65	Subtract 0.10 during simulation hours 17-20
Severe	0.50	Subtract 0.15 during simulation hours 16-21

The same seed reproduces the same batched demand and supply tensors in the pinned CUDA environment. Accepted reproduction uses seeds 50001-50003; the documented-seed-disjoint protocol uses the separate seeds 81001-81010.

The simulator parameters are reproducible, but reproducibility is not calibration. Their provenance classification is:

Parameter family	Classification	Basis and limitation
Seven locality names	Proposal-constrained	Defines the accepted case-study vocabulary; names do not supply electrical data
OCHA/HDX boundaries and Natural Earth rivers	Sourced	Presentation context only; not topology, demand, or controller input
700 MW peak, locality weights, and sector shares	Illustrative	Locked research assumptions, not utility measurements
Hourly sector profiles and +/-5% demand noise	Illustrative/derived	Deterministic profile construction plus seeded synthetic variation
Mild, Moderate, and Severe shortage fractions/windows	Proposal-constrained/illustrative	Reproducible shortage-only test conditions, not historical generation traces
Supply tied to total demand and capped below demand	Illustrative	Creates controlled shortages but a structural demand-supply relationship
PPO actor and Fixed Rotation schedule	Sourced/derived	Hash-bound accepted archive lineage and audited clean implementation

A realistic calibration would require feeder/locality demand, independent generation and supply constraints, electrical-network topology, critical-facility loads and priorities, outage and restoration histories, operating and switching rules, weather/calendar effects, and validated asset-to-locality mappings. Matched scenarios isolate controller differences within this generator; they do not remove misspecification shared by both controllers. The scientific case is therefore synthetic and Khartoum-inspired.

Shortage variants are intentionally coupled. For each seed, the generator first creates one severity-neutral `ScenarioBaseDraws` batch containing demand-uniform and supply-uniform tensors. That base batch is reused unchanged for Mild, Moderate, and Severe; a pure transformation applies only the level's shortage fraction and additional shortage window. For a fixed `(seed, batch_index)` pair, demand perturbations and the supply-noise draw are therefore identical across all three variants. The design contains 96 base-draw clusters, each observed under three shortage levels, rather than 288 independent random draws.

Controller decisions and dispatch

At every hour, each controller chooses one service tier for each locality:

- Off: request no load.
- Critical Only: request Hospital, Water, and Bakery load.
- Full Service: request all five sectors.

The simulator concatenates the requested critical and non-critical loads. If total requested power exceeds available supply, one proportional scale factor is applied to every requested load. This guarantees that simulated served power does not exceed the global supply pool, but it does not add a second within-hour critical-first dispatch stage. Critical priority affects the selected tier, not feeder-level hospital restoration after oversubscription.

Fixed Rotation

Fixed Rotation divides the day into three-hour blocks. In each block, three consecutive localities receive Full Service and the remaining localities are Off. The starting locality advances by three positions at each block and wraps around the seven-locality order. The rule does not observe current demand, supply, critical-service need, or cumulative fairness. It is supply-blind, limited to Off and Full Service, and cannot select Critical Only. This makes it transparent and proposal-aligned but weaker than a supply-aware critical-preserving heuristic; the comparison must not be generalized to every reasonable rule-based policy.

In the accepted reproduction, its critical-service ratios are 43.27%, 43.27%, and 41.35% under Mild, Moderate, and Severe shortages; utilization is 54.26%, 68.68%, and 90.73%. The rise in utilization as shortage worsens is a denominator effect because utilization divides by available usable power, not evidence that the schedule adapts. Mean unused available energy is 2,158.7 MWh for Fixed Rotation versus 150.2 MWh for PPO, so part of the utilization contrast is attributable to the baseline's design.

PPO

PPO [2] receives a causal 32-value observation at each hour:

- four global values: sine and cosine of hour, supply divided by demand, and supply divided by the 700 MW system peak; and
- four values for each of seven localities: normalized current demand, current critical-sector share, cumulative service ratio, and previous service tier divided by two.

The inference-only actor has two 512-unit tanh layers and 2,187 output logits. The 2,187 outputs represent all 3^7 joint locality actions. Deterministic argmax inference selects one joint action, which is decoded into seven service tiers using the pinned little-endian base-3 encoding. The actor is loaded from a hash-verified, pickle-free NumPy artifact and runs on `cuda:0` in inference mode.

This release reproduces the frozen actor's architecture, weights, actions, and evaluation. It does not include optimizer state and does not claim bitwise reproduction of the original training run. The joint-action head grows as 3^N: the accepted seven-locality actor has `3^7 = 2,187` outputs, so the architecture does not scale gracefully to a much larger locality count.

Historical PPO training record

The strict, model-manifest-bound record is `data/model/training-provenance.yaml`, SHA-256 `d2568bce26e9221afa8c266113a704f6fbf2370d3bc949f5aa6c72a7411ba1a8`. It records the 32-value input, two 512-unit tanh layers, separate actor/critic encoders, 2,187 action logits, optimizer settings, workload, seeds, validation, selection, selected-run hardware and convergence, six archive code-source bindings, and three run/metric/experiment record bindings. Its field map distinguishes directly recorded facts, source-reconstructed values (including named PyTorch defaults), arithmetic-derived values, and unavailable historical environment facts. Behavior-cloning initialization supplied the starting PPO actor from 32,768 teacher samples; it is initialization, not a third controller.

No behavior-cloning-only ablation was run, so this comparison cannot isolate the incremental effect of PPO fine-tuning; the initialization is checkpoint lineage, not an independently evaluated comparator.

For the selected seed-43 run, PPO ran 600 iterations with 256 complete 24-hour episodes per iteration: 153,600 episodes and 3,686,400 transitions. Each iteration used four epochs, 2,048-sample minibatches, 12 optimizer updates, `gamma=0.99`, `GAE lambda=0.95`, learning rate `0.0001`, clip epsilon `0.2`, value coefficient `0.5`, normalized entropy coefficient `0.001`, and separate actor/critic maximum gradient norms of `0.5`. The three-seed candidate experiment totaled 1,800 iterations, 460,800 episodes, and 11,059,200 transitions. Its historical promotion gates required all runs to be safe and finite, reward and seed-win margins, bounded worst-seed regression, and acceptable throughput. Those gates are historical selection facts, not independent headline evidence. Adam optimized historical PPO parameters; the selected checkpoint was iteration 510 after 6,120 updates. The recorded runtime, hardware, and convergence series apply only to selected seed 43. CUDA accelerated computation. No separate online mathematical dispatch optimizer runs in the desktop or documented-seed-disjoint comparison.

Reward

The accepted hourly reward combines critical service, utilization, fairness, switching, and oversubscription:

```
r_t = (5 C_t + 2 U_t + 2 J_t - 0.25 S_t - 3 O_t) / 9
```

Here, `C_t` is the critical-service ratio, `U_t` is utilization, `J_t` is Jain fairness [3] of cumulative locality service ratios, `S_t` is the fraction of localities that changed tier, and `O_t` is positive requested-power excess divided by demand. Switching is zero at the first hour. Episode reward is the sum of the 24 hourly rewards.

For locality `i`, let `x_i,t` be cumulative served energy divided by cumulative demand through hour `t`. The hourly fairness term is:

```
J_t = (sum_i x_i,t)^2 / (7 sum_i x_i,t^2 + 1e-8)
```

Using cumulative ratios makes fairness depend on service history rather than only the current hour.

The reward is a research design choice, not an economic valuation of electricity or service interruption. Because it directly includes critical service, utilization, fairness, switching, and oversubscription, it is publicly labeled Training-objective score (diagnostic) rather than treated as independent outcome evidence.

Outcome definitions and signs

All ratios are unitless and remain in the interval [0, 1] unless a paired difference is being reported.

Outcome	Definition	Preferred sign for `PPO - Fixed Rotation`	Role
Training-objective score (diagnostic)	Sum of 24 accepted hourly rewards	Positive	Diagnostic
Critical-service ratio	Critical-sector energy served / critical-sector demand	Positive	Supporting

Outcome	Definition	Preferred sign for `PPO - Fixed Rotation`	Role
Utilization	Mean hourly served power / `min(supply, demand)`	Positive	Primary
Jain fairness	Mean hourly Jain fairness of cumulative locality service ratios	Positive	Primary
Served energy	Sum of dispatched energy	Positive	Supporting
Curtailed energy	Demand minus served energy	Negative	Energy complement
Switching rate	Mean hourly fraction of localities changing tier; hour zero is zero	Negative	Secondary
Oversubscription	Mean positive requested-power excess / demand	Negative	Diagnostic
Power-balance violation rate	Fraction of hours served power exceeds supply by more than 0.001 MW	Non-positive	Safety
Critical-outage locality-hours	Locality-hours with critical-sector service below 99%	Negative	Primary
Demand and available energy	Scenario inputs summed over the episode	Not comparative	Input

Because every time step is one hour, power sums in the energy metrics have the numerical unit MWh. Critical outage locality-hours can range from 0 to 168 in one episode: 24 hours multiplied by seven localities.

Canonical aggregation rules

Every public metric has one calculation level and one aggregation rule:

- The episode-pooled critical-service ratio is total critical energy served divided by total critical energy demanded within one 24-hour episode. Controller aggregates are arithmetic means of those episode ratios; they do not pool energy across episodes.
- The mean hourly supply-utilization ratio is first calculated within an episode as the mean of 24 hourly `served / min(supply, demand)` values, then averaged across episodes.
- The mean hourly cumulative-locality Jain fairness is first calculated within an episode as the mean of 24 hourly fairness values over cumulative locality service ratios, then averaged across episodes.
- Reward is the 24-step sum per episode. Energy and locality-hour outcomes are 24-step sums per episode. Switching, oversubscription, and power-balance violation rate are hourly means per episode.

For each controller in the accepted reproduction, the aggregate is the equally weighted arithmetic mean of the 288 episode values. Equivalently, it is the equally weighted mean of the nine cell means because every cell contains 32 episodes. The documented-seed-disjoint aggregate is the same canonical calculation over 960 episodes: 10 seeds x 32 base draws x 3 shortage variants. Paired inference always uses within-scenario `PPO - Fixed Rotation` episode differences while keeping the three shortage variants of each base draw in the same resampled cluster.

Accepted reproduction design

The evaluation is a deterministic, within-scenario paired comparison.

Design element	Locked value
Shortage levels	Mild, Moderate, Severe

Design element	Locked value
Seeds per shortage level	50001, 50002, 50003
Cells	9
Episodes per cell	32
Independent (seed, batch_index) base-draw clusters	96
Shortage variants per base draw	3
Matched scenario conditions	288
Controllers evaluated per scenario	2
Controller-scenario evaluations	576
Episode horizon	24 one-hour steps

For each shortage-level and seed cell, the generator creates one 32-episode Scenario batch. The exact same immutable-reference Scenario object is passed first to Fixed Rotation and then to PPO; the two controllers therefore receive identical demand and supply tensors. Metrics remain on the GPU until the final serialization boundary. After the one packed host transfer, the canonical artifact builder deterministically materializes episode rows and computes the small row, cell, and aggregate summaries in NumPy/Python. Every stored paired difference is defined as:

```
delta = PPO - Fixed Rotation
```

This sign convention means positive values favor PPO for higher-is-better metrics, while negative values favor PPO for lower-is-better metrics.

The estimand is the equally weighted mean paired difference across the nine shortage-level and seed cells. Equal weighting is exact here because every cell contains 32 variants. The 288 rows are valid matched controller comparisons, but their 96 base draws are the sampling clusters. The estimand does not estimate the real-world prevalence of any shortage level or seed-defined scenario family.

The 50001-50003 seeds were not used for PPO gradient updates or periodic validation, but the archived promotion workflow reused them during model selection. Consequently, this design supports historical parity and accepted reproduction only. It is not described as an independently custodied final test.

Paired uncertainty calculation

Confidence intervals are calculated from the regenerated per-episode evidence. The accepted frozen artifact contains cell summaries rather than episode rows and therefore cannot support confidence intervals, p-values, or episode-level win rates.

For each inferential metric, the 288 paired differences are arranged as a [3 shortage levels, 3 seeds, 32 base draws, metric] tensor. Each of 10,000 bootstrap replicates samples 32 base-draw indices with replacement within each seed. The same sampled index is reused for that base draw's Mild, Moderate, and Severe variants, so correlated shortage variants stay together. The replicate statistic is the equally weighted mean over shortage levels, seeds, and sampled base draws. The reported 95% interval uses an explicit CuPy CUDA sort followed by linear interpolation at the 2.5th and 97.5th percentile ranks.

This is a deterministic seed-stratified paired cluster percentile bootstrap [4]. It holds the three shortage levels and three seed strata fixed; it does not resample shortage levels or seeds. This preserves the intended dependence and stratification but limits the intervals to the locked design.

The bootstrap seed is 20260815. Differences and reductions use float64. PyTorch [5] CUDA tensors are exposed to CuPy [6] through zero-copy DLPack; sampling and percentile calculation stay on cuda:0. Mean, lower, and upper statistics return through one stacked DLPack transfer, and the evaluator performs one final

packed full numeric evidence-array transfer for canonical JSON serialization. Boundary validators may synchronize scalar invariant flags; they do not move the simulation, metric, forecast, or bootstrap arrays or calculations to CPU.

These intervals describe resampling variability of 96 base-draw clusters conditional on nine synthetic cells. They are neither p-values nor field-causal estimates and do not establish performance on Sudan's real grid.

The reproduced critical-service estimand first calculates a ratio for each episode and then averages those episode ratios. The accepted cell summaries used pooled critical energy. Their maximum cell difference is `0.000001268461346626 2817`, which is below the locked `0.000002` tolerance. The new rows should therefore be understood as a source-compatible episode-level reproduction, not reconstructed historical episode data.

Documented-seed-disjoint robustness, sensitivity, and forecast design

The holdout supplement uses seeds 81001-81010 with 32 base draws per seed. Crossing these 320 independent base draws with all three shortage levels gives 960 matched scenario conditions and 1,920 controller-scenario evaluations. The two controllers reuse the exact same validated scenario within every pair, and device-side mutation guards reject detected writes to demand or supply. The deterministic seed-stratified paired cluster bootstrap uses 10,000 replicates and seed 20260826; it resamples 32 base-draw indices within each of 10 seed strata and keeps all three shortage variants together. Result classification is descriptive: sign-neutral integrity accepts favorable, adverse, or ambiguous intervals.

Sensitivity uses the same base draws. Each of seven locality weights and five sector shares is multiplied separately by 0.75 and 1.25, then its affected vector is renormalized to preserve the original total. This gives 24 one-factor-at-a-time +/-25% variants, or 25 parameterizations when the unperturbed base is included. Every variant is evaluated over the same 960 matched scenarios. For each primary outcome the supplement stores the variant paired effect and the difference-in-differences relative to the base paired effect. This is a bounded local sensitivity study, not a proof of global robustness.

All 144 paired and difference-in-differences intervals are descriptive, pointwise 95%, and unadjusted for multiple comparisons; no family-wide significance claim follows. Every intervention holds the base available-supply trace fixed and changes only its named demand-allocation vector.

On the 320 documented-seed-disjoint synthetic demand traces, forecast evaluation aligns both methods to target hours 3-23. The three-hour trailing mean uses only hours `t-3` through `t-1`; one-hour persistence uses hour `t-1`. The supplement stores per-seed sufficient statistics and recomputable pooled MAE and RMSE for 235,200 locality-sector predictions and 6,720 system-total predictions per method. At each one-hour target, residuals are in MW. Locality-sector statistics pool absolute and squared cell residuals across episode x target x 7 x 5; system-total error is calculated after summing the 35 signed cell residuals. SAE is in MW, SSE in MW squared, and pooled MAE/RMSE in MW. These values compare two synthetic display methods and do not establish real-world forecast accuracy.

Causal display-only demand projection

The desktop presentation can display a one-hour-ahead demand projection. For each locality and sector, the default projection is the mean of the preceding three observed hours. A target is never included in its own input window, and changes to future observations cannot affect an earlier projection. The operation remains on the input PyTorch device.

With a three-hour window, current hours 2 through 22 display locality projections for hours 3 through 23. The first two hours have insufficient history and the final hour has no next-hour target, so the UI shows no projection for those edge cases.

This projection is intentionally simple. It demonstrates causal prediction logic for decision-support presentation, but it is not consumed by the PPO observation, simulator, reward, or controller comparison. Its synthetic-trace MAE and RMSE are evaluated against persistence in the robustness supplement; no real-world forecast-accuracy claim is made.

Reproducibility and provenance

The proposal, accepted source benchmark, source checkpoint, clean inference artifact, historical training record, accepted reproduction, and documented-seed-disjoint robustness supplement are bound by SHA-256 manifests. The key final evidence hashes are:

Artifact	SHA-256
Accepted proposal PDF	`3432e64a21ab9741bf47646fd0f4f561522ddfd81d4005deb892d38672c65e48`
Frozen aggregate benchmark	`3d720ba87a747a5780119decd7144b4051fc36fc72ab50f229f5750ad749fd15`
PPO inference artifact	`3e4a7ce44267dd1cf3bbde7e77abafb8f31fdea77ece63acda7be17fa3198ad5`
Accepted paired reproduction	`7f2009c4f7efbbedf098069ced9218addec51d55a18b969e6c54433027020af5`
Documented-seed-disjoint robustness supplement	`fabb5a2f13ab31cd51f641f4e6a6db3cce8a28a6e2c4fa144612f634bad0beb1`
PPO training provenance YAML	`d2568bce26e9221afa8c266113a704f6fbf2370d3bc949f5aa6c72a7411ba1a8`
Project source manifest	`a8377ee96b8c7742f0794776b5752a9553abd0f766b78e457565428d612463e6`

The reviewer contract has four gates: portable development/CPU CI; the mandatory presentation-GPU gate for source/runtime/evidence; the strict post-build `release_artifact` gate; and the optional source-archive parity gate. The normal package build does not require the sibling research archive.

Run the gates from the repository root after installing the locked environment:

```
uv sync --locked --extra dev --extra docs --offline

# Portable development/CI gate
uv run --no-sync pytest -m "not gpu and not source_archive and not release_artifact"

# Mandatory presentation-GPU gate
uv run --no-sync gridguard-doctor
uv run --no-sync pytest -ra -m "not source_archive and not release_artifact"
uv run --no-sync gridguard-evaluate --verify-evidence
uv run --no-sync ruff check .

# Build first, then inspect the final release tree with zero skips.
.\scripts\build-presentation.ps1
uv run --no-sync pytest -ra -m "release_artifact"

# Optional source-archive parity gate
.\scripts\build-presentation.ps1 -StrictSourceParity `
  -SourceRepositoryPath ..\aipower_rl `
  -SourceCheckpointPath ..\aipower_rl\artifacts\checkpoints\best.pt
```

Both mandatory selected suites require zero skips. Strict source parity validates the two explicit paths and runs the `source_archive` tests; it is not a runtime or normal-build dependency.

The paired artifact can be independently regenerated to a temporary file without overwriting the committed evidence. The command refuses to target the trusted artifact and accepts the reproduction only when its bytes produce the pinned SHA-256:

```
$reproduction = Join-Path $env:TEMP "gridguard-paired-benchmark.json"
uv run gridguard-evaluate --reproduce-paired --output $reproduction
```

The required output digest is `7f2009c4f7efbbedf098069ced9218addec51d55a18b969e6c54433027020af5`.

The documented-seed-disjoint supplement has the same overwrite protection and byte-reproduction requirement:

```
$robustness = Join-Path $env:TEMP "gridguard-robustness-supplement.json"
uv run gridguard-evaluate --reproduce-robustness --output $robustness
```

Evidence verification deliberately separates scientific integrity from result direction. `validate_paired_benchmark_payload` performs neutral integrity checks: closed schema, matched inputs, finite physical bounds, energy identities, row deltas, recomputed summaries, and a parity status consistent with the stored errors. It can therefore validate a well-formed adverse or inconclusive artifact. `assert_paired_benchmark_integrity` adds locked provenance, a direct cross-check against the hash-pinned frozen benchmark, and passing 18-cell parity without checking the sign of any interval. The robustness verifier independently reconstructs protocol counts, controller-minus-baseline means, difference-in-differences, forecast sufficient-statistic identities, and result classifications. No favorable-result release gate exists.

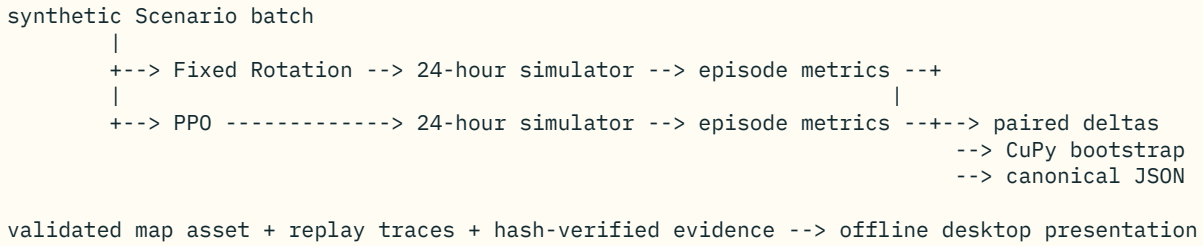
Limitations and threats to validity

- All demand and supply traces are synthetic and are not calibrated against utility records.
- The illustrative supply trace is constructed partly from synthetic demand; matched controller scenarios do not remove this shared generator assumption.
- The seven localities share one abstract supply pool; the model omits network topology, feeder limits, losses, voltage, frequency, protection, restoration, and operator constraints.
- Proportional dispatch scales critical and non-critical requests together when total requests exceed supply; it does not model feeder-level within-hour critical restoration priority.
- The one-hour resolution cannot represent sub-hour switching or transient grid behavior.
- Fixed Rotation is supply-blind and limited to Off and Full Service. The study does not establish superiority over a supply-aware critical-preserving heuristic, every optimization method, or every learned controller.
- Performance relative to the global optimum is unknown; no episode-level optimization or valid upper-bound comparator is included.
- The PPO joint-action head has 3^N scaling (`3^7 = 2,187` here), limiting architectural scalability beyond the accepted seven-locality scope.
- Both designs remain conditional on fixed synthetic seed strata and shortage construction, even though the documented-seed-disjoint supplement expands coverage to ten seeds and adds local sensitivity.
- Power-balance compliance is an invariant of this simulator, not evidence of real-grid stability or operational safety.
- The 99% critical-outage threshold and reward weights are explicit research assumptions.
- The evaluated operating domain is shortage-only: Mild, Moderate, and Severe. It does not cover surplus, normal operation, recovery, or restoration transients.
- The source-pinned administrative boundaries and Nile linework are presentation context only; they are not electrical-network topology and do not improve the simulator's external validity.
- The causal projection is evaluated only on synthetic demand against persistence; it has no real-data validation and remains excluded from controller decisions.

Implementation

System overview

GridGuard Khartoum is an offline Python 3.12 desktop application and reproducible CUDA simulation package. It separates four concerns: immutable domain contracts, deterministic scenario and controller execution, audited evidence generation, and presentation-only UI rendering.



The two controller paths share the exact same `Scenario` object. The desktop presentation may replay a selected scenario, but its Results view reads committed evidence rather than silently recalculating the headline claim.

Module map

Path	Responsibility
`src/gridguard/domain.py`	Locality, sector, shortage, service-tier, controller, scenario, result, and metric contracts
`src/gridguard/claim_contract.py`	Canonical labels and bounded limitations for reviewer-facing scientific claims
`src/gridguard/scenarios.py`	Severity-neutral stochastic base draws and pure shortage-level scenario transformations
`src/gridguard/controllers.py`	Shared controller protocol, Fixed Rotation rule, and PPO adapter
`src/gridguard/simulation.py`	Causal 32-value observation, tier dispatch, reward, and immutable result traces
`src/gridguard/metrics.py`	Per-episode proposal outcomes on the source tensor device
`src/gridguard/policy.py`	Strict PPO actor schema, artifact verification, CUDA load, and deterministic inference
`src/gridguard/statistics.py`	CuPy stratified paired bootstrap through DLPack
`src/gridguard/evaluation.py`; `src/gridguard/paired_inference.py`	Exact pairing, evidence serialization/verification, frozen parity, and independent CUDA endpoint reconstruction
`src/gridguard/robustness.py`; `src/gridguard/robustness_inference.py`; `src/gridguard/robustness_provenance.py`	Holdout/sensitivity/forecast evaluation, reconstructable inference inputs, strict artifact verification, and non-circular trust anchors
`src/gridguard/forecast.py`	Causal trailing-window demand projection for display only
`src/gridguard/gpu.py`	Fail-closed CUDA, PyTorch, CuPy, driver, and DLPack preflight
`src/gridguard/provenance.py`	Pinned source-manifest and artifact verification
`src/gridguard/resources.py`	Safe source-checkout or installed-bundle resource resolution
`src/gridguard/ui/map_asset.py`	Provenance verification, closed GeoJSON validation, WGS84 projection, and locality keyboard graph
`src/gridguard/ui/app.py`	Desktop entry point and fail-closed startup sequence
`src/gridguard/ui/window.py`; `src/gridguard/ui/window_components.py`	1366 x 768 shell, accessible page stack/navigation, shortcuts, and persistent trust status
`src/gridguard/ui/presentation_model.py`	Strict projection of the documented-seed-disjoint robustness supplement into presentation data
`src/gridguard/ui/map_view.py` and `src/gridguard/ui/map_items.py`	Interactive `QGraphicsView` map and redundant service-state rendering
`src/gridguard/ui/live_models.py`; `src/gridguard/ui/live_runner.py`; `src/gridguard/ui/live_worker.py`	Validated timeline contracts, shared-scenario CUDA execution, one display transfer, and off-GUI-thread worker signals
`src/gridguard/ui/live_components.py`; `src/gridguard/ui/live_controls.py`; `src/gridguard/ui/live_visualization.py`; `src/gridguard/ui/live_page.py`	Stable focused widgets for controls, aggregate results, synchronized map/timeline replay, and locality details
`src/gridguard/ui/overview_page.py`	Scope, protocol, and locality overview
`src/gridguard/ui/results_page.py`	Artifact-derived controller comparison and paired intervals
`src/gridguard/ui/methodology_page.py`	In-application method and limitation summary
`src/gridguard/ui/components.py` and `src/gridguard/ui/theme.py`	Reusable accessible cards, metric formatting, and visual theme
`src/gridguard/cli.py`	`gridguard-doctor` and evidence-verification entry points

Domain and tensor contracts

The public vocabulary is encoded directly as enums. There are seven localities, five sectors, three shortage levels, and three service tiers. `Scenario` tensors have shapes:

```
demand: [batch, 24, 7, 5]
supply: [batch, 24]
```

Simulation traces have shapes:

```
service_tiers: [batch, 24, 7]
requested:     [batch, 24, 7, 5]
served:        [batch, 24, 7, 5]
reward:        [batch, 24]
```

Constructors reject invalid dimensions, mismatched devices or dtypes, negative power, non-finite values, invalid tiers, and inconsistent energy identities. Frozen dataclasses limit accidental reference reassignment, and the simulator does not mutate the scenario tensors.

GPU-first execution

The presentation and benchmark require `cuda:0`; there is no silent CPU fallback. PyTorch runs scenario generation, the 24-hour batched simulator, PPO inference, and metric calculation. CuPy runs the bootstrap sampling and percentiles. The large arrays remain on the GPU.

`run_gpu_preflight()` checks:

- PyTorch 2.8.0 with its CUDA 12.8 build;
- at least one visible NVIDIA device;
- CuPy in the supported range `>=13.6,<16` with a CUDA 12 runtime;
- an NVIDIA driver new enough for the CuPy runtime; and
- coherent zero-copy PyTorch/CuPy sharing by mutating a Torch CUDA tensor and observing the updated values through a CuPy DLPack view.

Any failed check stops launch or evaluation with an actionable error. Small unit tests can inject a CPU device for local logic checks, but the mandatory presentation-GPU gate runs the complete source/runtime suite on the target laptop and fails closed when CUDA is unavailable.

PPO artifact and inference

The desktop does not deserialize the source training checkpoint. A dedicated exporter extracts only the actor tensors into `data/model/ppo-policy.npz`. Runtime loading uses `numpy.load(..., allow_pickle=False)` and accepts exactly six float32 tensors with locked names and shapes.

Layer	Shape
First actor encoder weight	`[512, 32]`
First actor encoder bias	`[512]`
Second actor encoder weight	`[512, 512]`
Second actor encoder bias	`[512]`
Joint-action output weight	`[2187, 512]`
Joint-action output bias	`[2187]`

`data/model/manifest.json` binds the artifact to the accepted source checkpoint, source revision, and `data/model/training-provenance.yaml`. Before loading CUDA parameters, the verifier checks the externally pinned SHA-256, file size, schema, architecture metadata, tensor set, dtype, shape, finiteness, and exact training-record hash. The loaded actor is moved to `cuda:0`, frozen, placed in evaluation mode, and restricted to deterministic argmax actions.

The strict YAML record exposes the historical configuration reviewers asked for: the 32 -> 512 -> 512 -> 2,187 network; behavior-cloning initialization; Adam/AdamW settings; validation and selection seeds; selected iteration 510; selected-run hardware and convergence; six code-source bindings; and three run/metric/experiment record bindings. The selected seed-43 run used 600 PPO iterations, 153,600 episodes, and 3,686,400 transitions. The three-seed candidate experiment totaled 1,800 iterations, 460,800 episodes, and 11,059,200 transitions. Its historical promotion gates are selection facts, not independent headline evidence. A strict field map distinguishes directly recorded, source-reconstructed, arithmetic-derived, and unavailable historical environment facts. The record intentionally omits optimizer state and raw runs and makes no bitwise-retraining claim. Adam was the historical parameter optimizer; CUDA is the computation platform, not an online dispatch optimizer.

Scenario, dispatch, and metrics pipeline

`generate_scenario_base_draws()` creates one severity-neutral stochastic batch directly on the requested PyTorch device. A device-local seeded generator supplies the demand and supply uniform draws once. `scenario_from_base_draws()` then applies the selected shortage fraction and window as a pure transformation. The evaluator reuses the exact same `ScenarioBaseDraws` references across Mild, Moderate, and Severe, making cross-severity dependence explicit rather than relying on re-seeding side effects. The compatibility `generate_scenarios()` path composes those two steps. The simulator then iterates the 24 hours while keeping cumulative demand, cumulative served energy, and previous tiers on the same device.

At each hour:

1. Build the causal 32-value observation from current and accumulated information.
2. Ask the selected controller for seven integer tiers.
3. Reject malformed controller output before dispatch.
4. Convert tiers to requested sector loads.
5. Scale all requested loads proportionally if requests exceed available supply. This preserves power balance but scales critical and non-critical sectors together; it is not a second critical-first dispatch stage.
6. Update cumulative state and calculate the accepted reward.
7. Append immutable tensor traces for final metric calculation and presentation replay.

The metric module calculates one device-resident value per episode. Result and metric contracts, together with invariant tests, enforce `served + curtailed = demand`, verify served energy cannot exceed demand or availability, and report any hour exceeding the 0.001 MW power-balance tolerance.

The canonical aliases on `EpisodeMetrics` make aggregation semantics reviewable in code: `episode_pooled_critical_service_ratio`, `mean_hourly_supply_utilization_ratio`, and `mean_hourly_cumulative_locality_jain_fairness`. Controller aggregates are equally weighted arithmetic means of episode metrics. The accepted reproduction has 288 episodes and nine cells; the documented-seed-disjoint robustness supplement has 960 episodes and 30 shortage-by-seed cells. Every cell contains 32 episodes.

Accepted reproduction evaluation and statistics

`evaluate_scenario_pair()` executes Fixed Rotation and PPO on the exact same `Scenario` object. The full evaluator first creates one batch of 32 base draws for each of three seeds, then reuses each batch across all three shortage transformations. It evaluates the resulting nine cells in the canonical shortage-major, seed-minor order. The three shortage variants at a given `(seed, batch_index)` therefore share one stochastic draw, so the design has 96 base-draw clusters. Crossing those draws with three shortage levels produces 288 matched scenario conditions, 288 paired rows, and 576 controller-scenario evaluations.

Episode metrics are stacked into `[3 shortage levels, 3 seeds, 32 episodes, 12 metrics]` cubes. On CUDA, the evaluator calculates `PPO - Fixed Rotation` in float64, selects eight inferential metrics, and retains the `[3, 3, 32, 8]` dependence structure. For each of 10,000 deterministic replicates, CuPy samples 32 cluster indices with replacement inside each seed and reuses those indices across all three shortage variants. The statistic then gives equal weight to shortage levels, seeds, and sampled clusters. Endpoint calculation uses an explicit CUDA sort and linear interpolation, `a + (b - a) * weight`. This fixes the operation order and avoids reliance on the library percentile helper while preserving the defined linear percentile. Mean, lower, and upper values return to PyTorch in one stacked DLPack transfer. Only one packed full evidence-array transfer returns the two metric cubes and bootstrap output to host memory. The canonical builder then materializes the 288 small row mappings and computes their cell/aggregate summaries in NumPy/Python for deterministic JSON serialization and validation; no controller, simulator, metric, or bootstrap computation falls back to CPU.

The resulting `data/evidence/paired-benchmark.json` is explicitly labeled accepted reproduction because seeds `50001-50003` participated in the archived promotion/model-selection process. It contains:

- all 288 controller-paired episode rows;
- nine cell summaries;
- aggregate controller means;
- paired mean differences and 95% percentile intervals;
- metric definitions and direction metadata;
- frozen-cell parity errors and tolerances;
- exact scenario counts, seeds, and bootstrap protocol;
- the 96-base-draw cluster structure and three shortage variants per draw;
- PPO and source checkpoint provenance; and
- explicit scientific limitations.

Serialization is canonical UTF-8 JSON with sorted keys, finite numbers only, compact separators, and a final newline. `validate_paired_benchmark_payload()` is deliberately neutral: it requires a closed schema, exact row ordering, unique scenario identities, matched input energy, recomputed row deltas, metric physical bounds and energy identities, recomputed cell and aggregate means, inference metadata, and a parity status consistent with the declared errors. It does not require PPO-favorable intervals or locked artifact identities, so adverse and inconclusive research outputs can still be validated honestly.

`assert_paired_benchmark_integrity()` calls the neutral validator, then requires locked provenance, passing parity, and a direct reload and cross-check of all 18 controller cell summaries from the hash-pinned frozen artifact. It does not inspect interval direction. `verify_paired_benchmark()` always verifies the external file hash and canonical bytes; supplying the frozen benchmark path activates this stronger sign-neutral integrity check. Both functions remain host-only so the documented portable gate can verify immutable structure, arithmetic, physics, provenance, and claims with CUDA hidden.
`verify_paired_inference_on_cuda()` is the separate mandatory `cuda:0` reconstruction of every

persisted cluster-bootstrap endpoint. Desktop preflight, `gridguard-evaluate --verify-evidence`, and paired reproduction invoke the host verifier and then that CUDA reconstruction; the portable verifier is not represented as release certification.

Documented-seed-disjoint robustness pipeline

`run_full_robustness_evaluation()` evaluates seeds 81001-81010, 32 base draws per seed, and all three shortage levels. That is 320 independent base draws, 960 matched scenario conditions, and 1,920 controller-scenario evaluations. The same generic bootstrap preserves the `[3, 10, 32, 3]` primary-outcome structure and uses 10,000 replicates with seed 20260826.

`ScenarioDemandParameters` provides a validated, immutable perturbation seam while preserving the default generator byte-for-byte. The robustness evaluator changes each of seven locality weights and five sector shares separately by -25% and +25%, renormalizes the affected vector, and replays the same base draws while reusing the base available-supply tensor exactly. The 24 one-factor-at-a-time variants plus the base give 25 parameterizations, 24,000 matched sensitivity scenarios, and 48,000 controller-scenario evaluations. The artifact stores paired effects and difference-in-differences for critical-outage locality-hours, fairness, and utilization.

Forecast statistics compare the three-hour trailing mean with one-hour persistence on identical targets 3-23. Counts, absolute-error sums, squared-error sums, pooled MAE/RMSE, and cross-aggregation identities are recomputed by the strict verifier. Simulation, inference, bootstrap, sensitivity, and forecast statistics remain on `cuda:0`; validity flags and all numeric evidence are packed into one final full-array `.cpu().numpy()` transfer. The Scenario shortage preflight and each controller action range check may synchronize one-element invariant flags so invalid inputs fail before dispatch; no scientific array computation moves to CPU. Canonical host assembly then writes `data/evidence/robustness-supplement.json`. Verification requires exact provenance bindings, protocol counts, controller-minus-baseline means, difference-in-differences arithmetic, forecast identities, result classifications, canonical bytes, and the external SHA-256. It never requires a favorable sign.

The holdout label proves only disjointness from every documented training, validation, and model-selection seed in `data/model/training-provenance.yaml`; it does not prove preregistration, independent custody, or the absence of undocumented activity. Sensitivity intervals are descriptive, pointwise 95%, and unadjusted for multiple comparisons. No behavior-cloning-only ablation was run, so this comparison cannot isolate the incremental effect of PPO fine-tuning.

Evidence provenance and fail-closed behavior

`data/evidence/source-manifest.json` is itself pinned in code. It contains exact IDs and hashes for eight local artifacts (proposal, frozen benchmark, PPO actor, PPO manifest, training YAML, accepted reproduction, robustness supplement, and the robustness trust-anchor source module) and 15 source-lineage files. Verification rejects a rewritten manifest even if its internal entries are self-consistent.

The exact bundled scientific source set verified by the current robustness artifact and copied into wheel/PyInstaller reviewer assets comprises `src/gridguard/controllers.py`, `src/gridguard/claim_contract.py`, `src/gridguard/domain.py`, `src/gridguard/evaluation.py`, `src/gridguard/forecast.py`, `src/gridguard/metrics.py`, `src/gridguard/paired_inference.py`, `src/gridguard/policy.py`, `src/gridguard/robustness.py`, `src/gridguard/robustness_inference.py`, `src/gridguard/scenarios.py`, `src/gridguard/simulation.py`, and `src/gridguard/statistics.py`.

`src/gridguard/robustness_provenance.py` is the separately bundled, non-circular trust anchor for that set; packaging derives its inputs from those declarations, not a duplicated handwritten list.

Additional safeguards include:

- duplicate JSON fields and non-finite JSON constants are rejected;
- resource paths must remain relative to the runtime root;
- model files must match both an external trust anchor and their manifest;
- evidence cell counts, controller labels, seeds, and scenario IDs are closed rather than inferred from arbitrary input;
- sign-neutral integrity validation is independent of whether an interval favors PPO;
- frozen parity cannot rely only on a self-declared status;
- the paired file must match SHA-256 `7f2009c4f7efbbedf098069ced9218addec51d55a18b969e6c54433027020af5`;
- the robustness supplement must match SHA-256 `fabb5a2f13ab31cd51f641f4e6a6db3cce8a28a6e2c4fa144612f634bad0beb1`; and
- the evaluator refuses devices other than `cuda:0`, bootstrap counts other than 10,000, or a protocol-specific bootstrap seed.

Display-only forecast

`causal_demand_projection()` accepts `[batch, hour, 7, 5]` demand tensors. Its default output is a one-hour-ahead trailing mean over the previous three hours. The function validates shape, dtype, finiteness, non-negativity, window, and horizon, preserves the input dtype and device, and uses `torch.inference_mode()`.

The live CUDA runner sums the sector projections into one value per locality and aligns them so current hours 2 through 22 display forecasts for hours 3 through 23. The first two hours lack a three-hour history and hour 23 has no next-hour target, so those frames display no projection. The robustness evaluator additionally compares this trailing mean with one-hour persistence over common synthetic targets 3-23 and stores recomputable locality-sector and system-total MAE/RMSE. The module, runner, UI labels, and tests enforce the boundary: the projection is displayed only, is not controller input, and has no real-data accuracy claim.

Offline administrative map

The bundled geographic layer is a small, reproducible derivative of two public datasets. `khartoum-localities.geojson` selects the exact OCHA/HDX Sudan COD-AB v03 ADM2 P-codes SD01001 through SD01007 under ADM1_PCODE=SD01. It retains source English and Arabic names, P-codes, Polygon/MultiPolygon rings and holes, and source centers used as label anchors. `khartoum-nile.geojson` contains clipped Natural Earth 1:10m White Nile, Blue Nile, and Nile centerlines. These layers are presentation context only, results remain synthetic, and the view is not electrical-network topology.

`tools/build_map_asset.py` verifies the upstream byte sizes and SHA-256 pins before selecting, normalizing, joining the recorded 258.07 m Blue Nile confluence gap, clipping rivers to the dissolved locality mask, and writing deterministic GeoJSON. `map-provenance.json` binds the two derived hashes to the source selectors and EPSG:3857 scene transform. Attribution and licensing are bundled in `src/gridguard/assets/map/THIRD_PARTY_DATA.md`.

At startup, a standard-library loader verifies provenance before parsing either GeoJSON file and rejects duplicate or unknown fields, non-finite coordinates, unsupported geometry, missing or duplicate identities, incorrect P-codes, invalid keyboard/neighborhood order, or coordinates outside the closed geographic contract. One fixed formula projects WGS84 to EPSG:3857, aspect-fits all boundaries, holes, anchors, and rivers into the 1000 by 720 scene, and gives Qt only projected coordinates. No GIS library or network request is required at

runtime. Keeping scenario and controller fields out of the assets prevents presentation geography from becoming scientific input.

Desktop presentation

`gridguard-desktop` starts the PySide6 application through `ui/app.py`. Before it creates the main window, startup runs the complete project preflight: GPU and DLPack checks, the pinned source manifest, accepted paired-evidence semantics and frozen parity, seed-disjoint robustness semantics and implementation-source bindings, PPO artifact verification, and a golden CUDA inference action. It then loads the pinned robustness supplement for the Results view and the bundled map. A GPU, evidence, model, resource, or map failure blocks the presentation instead of opening a partly trusted interface.

The main window is designed at 1366 x 768 and contains four pages: Overview, Live Scenario, Results, and Methodology. Ctrl+1 through Ctrl+4 select the pages and F11 toggles fullscreen. Controls expose accessible names and descriptions, accept keyboard focus, and use a visible blue focus outline. Dynamic run, hour, projection, and service-tier labels emit accessibility value or name-change notifications when their text changes.

The `QGraphicsView` map supports pointer selection against real compound locality paths, arrow-key navigation through explicit locality neighbors, wheel or plus/minus zoom, drag-to-pan, and R or `0` to reset the view. A synchronized locality list provides a persistent text target for the same seven selections. Service state is never communicated by color alone: fill, texture, symbol, text, tooltips, and the list distinguish Off, Critical Only, and Full Service. The map retains its administrative-context, synthetic-results, and no-electrical-topology notice.

The Live Scenario page accepts a shortage level and seed and starts a `QThread` worker so CUDA execution does not block the Qt event loop. The runner generates one `cuda:0` scenario, passes the exact same object through PPO and Fixed Rotation, and validates that every scenario and result tensor remains on `cuda:0`. Inputs are disabled while the worker is active, and the window refuses to close until the CUDA run has completed safely.

After both controllers finish, the runner packs hourly supply, 35 locality-sector demand values, seven next-hour locality projections, seven PPO tiers, 35 PPO served values, seven Fixed Rotation tiers, and 35 Fixed Rotation served values into one `[24, 127]` display tensor. It then appends five episode metrics for each controller to the same flattened transfer: served energy, critical-service ratio, fairness, curtailed energy, and power-balance violation rate. This is the UI runner's only bulk device-to-host transfer. The resulting immutable comparison must contain exactly 24 frames and one validated `LiveEpisodeSummary` for each named controller, with identical supply, system demand, locality demand, and projection values at every paired hour.

A table labeled Live demonstration presents Served energy, Critical service, Fairness, Curtailed energy, and Power-balance violations for PPO and Fixed Rotation from the selected episode. It is explicitly separate from frozen evidence. A controller selector and 24-hour slider let the presenter compare tier allocations on that same scenario. The selected-locality panel derives actual demand MW, served MW, critical-service ratio, and service ratios for Hospital, Water, Bakery, Residential, and Commercial from the transferred simulation values. It also labels the causal next-hour demand projection as display only and explicitly states that the value is not PPO input or evidence. Until a valid comparison arrives, the page displays an honest waiting state; it never fabricates replay data. Runtime errors clear the frames and aggregate table, disable replay controls, and remain visible to the presenter.

The Results page renders only the three primary values and paired intervals present in the documented-seed-disjoint robustness supplement. It shows power balance separately as safety and does not calculate replacement headline values. The Methodology page summarizes the accepted reproduction, documented seed disjointness, historical training record, locked design, and evidence limitations for presentation use.

Runtime resources and packaging

`runtime_root()` supports two layouts:

1. a source checkout containing `data/evidence/source-manifest.json`; or
2. an installed package containing a `_bundled` runtime tree.

The wheel configuration force-includes `data/evidence`, `data/model`, and the accepted proposal PDF. The map GeoJSON, provenance, attribution, and font resources are package data under `src/gridguard/assets`. The same safe resolver is used for source and installed execution, and path traversal outside the chosen root is rejected.

The application is designed for an offline Windows presentation. No evidence, map geometry, model weight, or replay dependency requires a network request after installation.

Verification strategy

The tests are organized by the same boundaries as the implementation:

Test group	What it proves
Domain and scenario tests	Shapes, vocabulary, seed determinism, shortage construction, and GPU generation
Controller and simulation tests	Fixed schedule, action validation, dispatch, reward, and golden traces
Metric tests	Energy identities, proposal definitions, sign conventions, and finite device-resident outputs
Policy tests	Safe artifact parsing, actor architecture, source parity, decoding, and real CUDA inference
Evaluation and statistics tests	Exact pairing, evidence roles, counts, bootstrap stratification, DLPack sharing, canonical evidence, and frozen parity
Robustness tests	Documented-seed-disjoint seeds, 24 sensitivities, difference-in-differences, forecast arithmetic, one transfer, and sign neutrality
Provenance and source tests	Pinned manifest, training YAML, source Git blobs, accepted benchmark migration, and tamper rejection
Forecast tests	Causality, common-target alignment, persistence comparison, shape, dtype, device preservation, and invalid-input rejection
Map and package tests	Source/derived hashes, closed GeoJSON, real compound geometry, river paths, keyboard metadata, wheel inclusion, and isolated install loading
Desktop UI tests	Startup-gate ordering, evidence projection, tabs, shortcuts, map interaction, and asynchronous live success/error states
GPU integration tests	Real PyTorch/CuPy/CUDA preflight on the presentation laptop

The portable command excludes GPU, source-archive, and built-release checks:

```
uv run --no-sync pytest -m "not gpu and not source_archive and not release_artifact"
```

The remaining contract has a mandatory presentation-GPU gate and strict post-build `release_artifact` gate:

```
uv run --no-sync gridguard-doctor
uv run --no-sync pytest -ra -m "not source_archive and not release_artifact"
uv run --no-sync gridguard-evaluate --verify-evidence
uv run --no-sync ruff check .
```

```
# Run only after the Windows package and release manifest are final.
uv run --no-sync pytest -ra -m "release_artifact"
```

The optional source-archive parity gate is an explicit lineage audit, not a package dependency. The normal package build does not require the sibling research archive. With that read-only archive and checkpoint available, run:

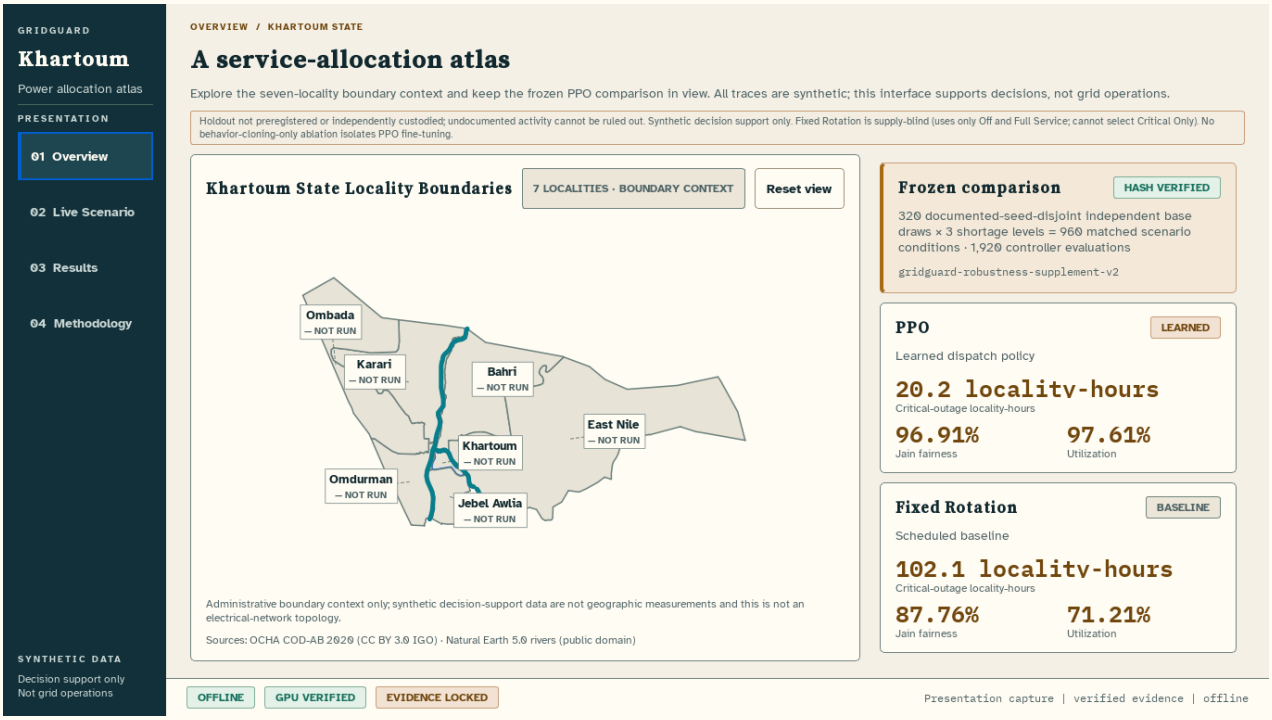
```
.\scripts\build-presentation.ps1 -StrictSourceParity `
  -SourceRepositoryPath ..\aipower_rl `
  -SourceCheckpointPath ..\aipower_rl\artifacts\checkpoints\best.pt
```

Strict mode validates both paths and includes `source_archive` tests. Normal builds clear ambient source-path variables, never probe the sibling repository, and require zero skips in both mandatory selected suites.

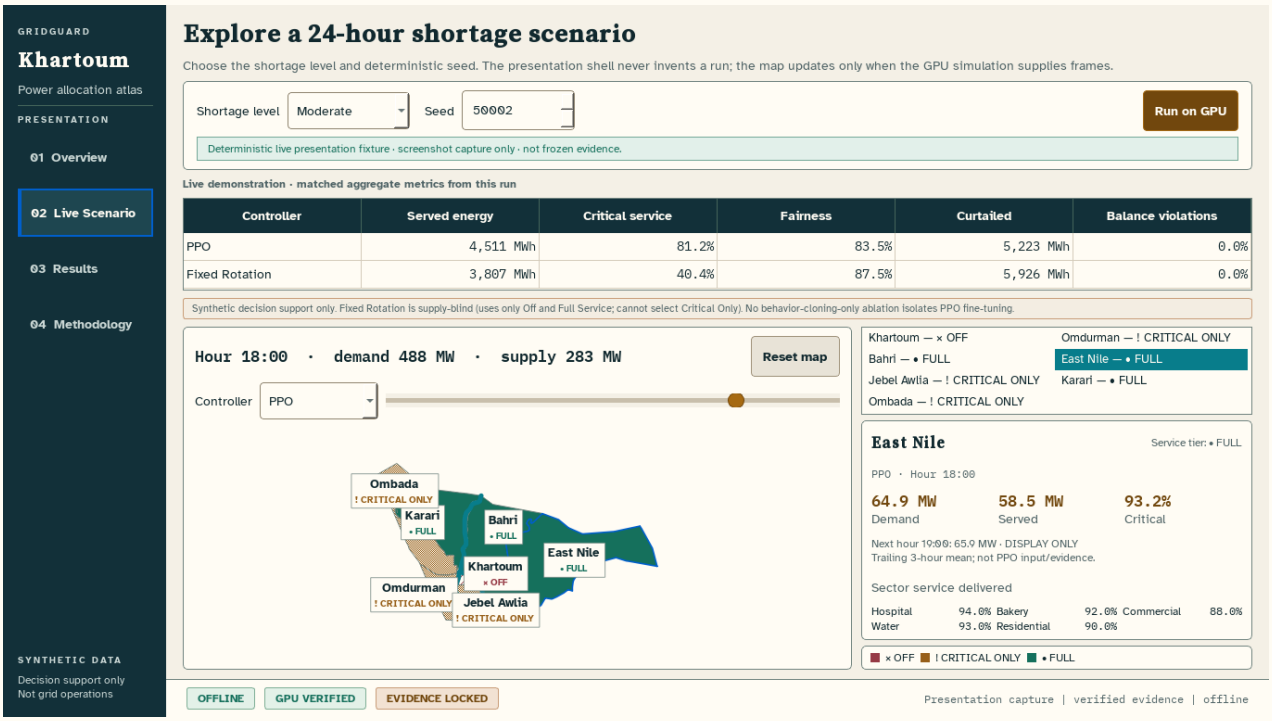
Deliberate implementation limits

- Training code, optimizer state, and raw training runs are not part of the clean presentation runtime; the strict YAML is a historical provenance record, not a retraining bundle.
- The simulator is a decision-support abstraction and has no hardware control interface.
- The map contains real administrative boundaries and river context but no feeders, substations, measured demand, service territories, or other electrical-network topology.
- NumPy/Python is restricted to post-transfer artifact parsing, deterministic assembly, and small host validation; simulation, inference, metrics, forecast, and bootstrap stay on the GPU. Scalar invariant checks may synchronize.
- `evaluation.py` and the robustness orchestration remain large. Only assurance-critical scenario-identity, persisted-input, reconstruction, and trust-boundary helpers were extracted; a broad late-stage refactor was deliberately deferred to avoid scientific regression.
- Fixed Rotation is supply-blind, uses only Off and Full Service, and cannot select Critical Only. The comparison neither covers a stronger supply-aware heuristic nor locates PPO relative to a global optimum.
- The result claim is fixed to the two named controllers, three shortage-only levels, and one frozen PPO actor. Surplus, recovery, normal operation, and restoration transients are excluded.
- The 3^N joint-action output has 2,187 logits at seven localities and is not presented as a scalable architecture for a national grid.

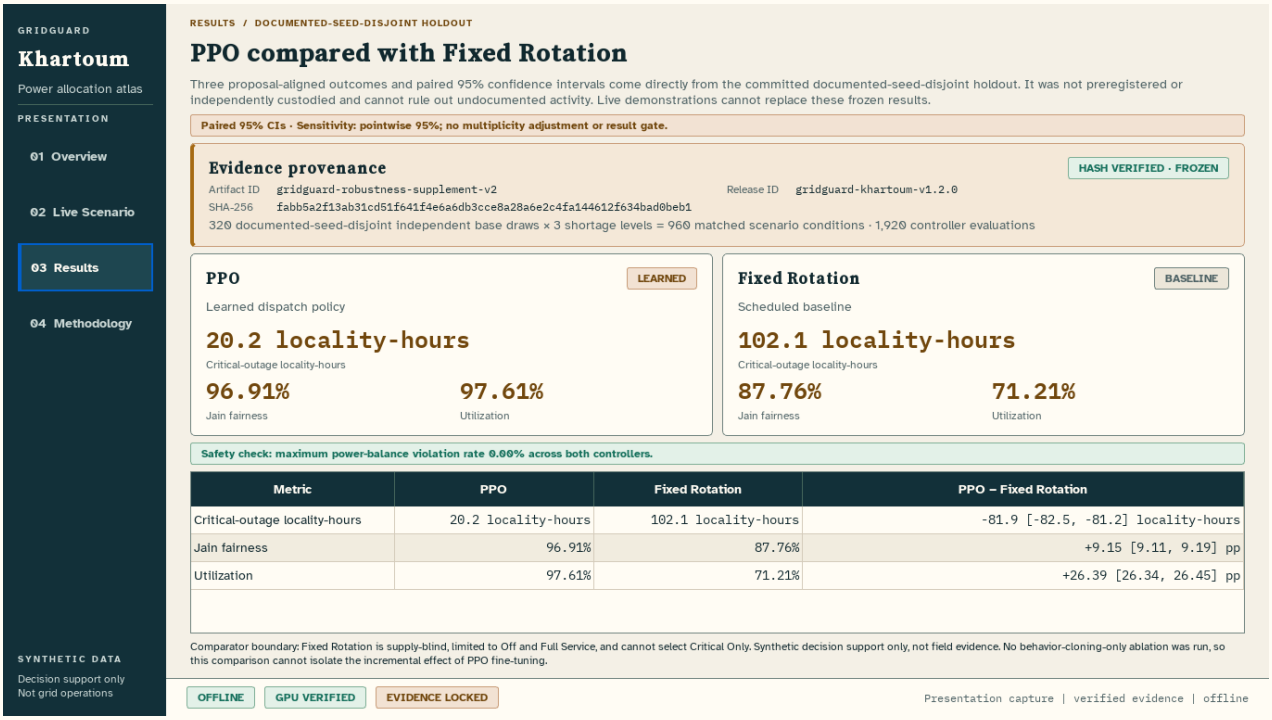
Desktop presentation



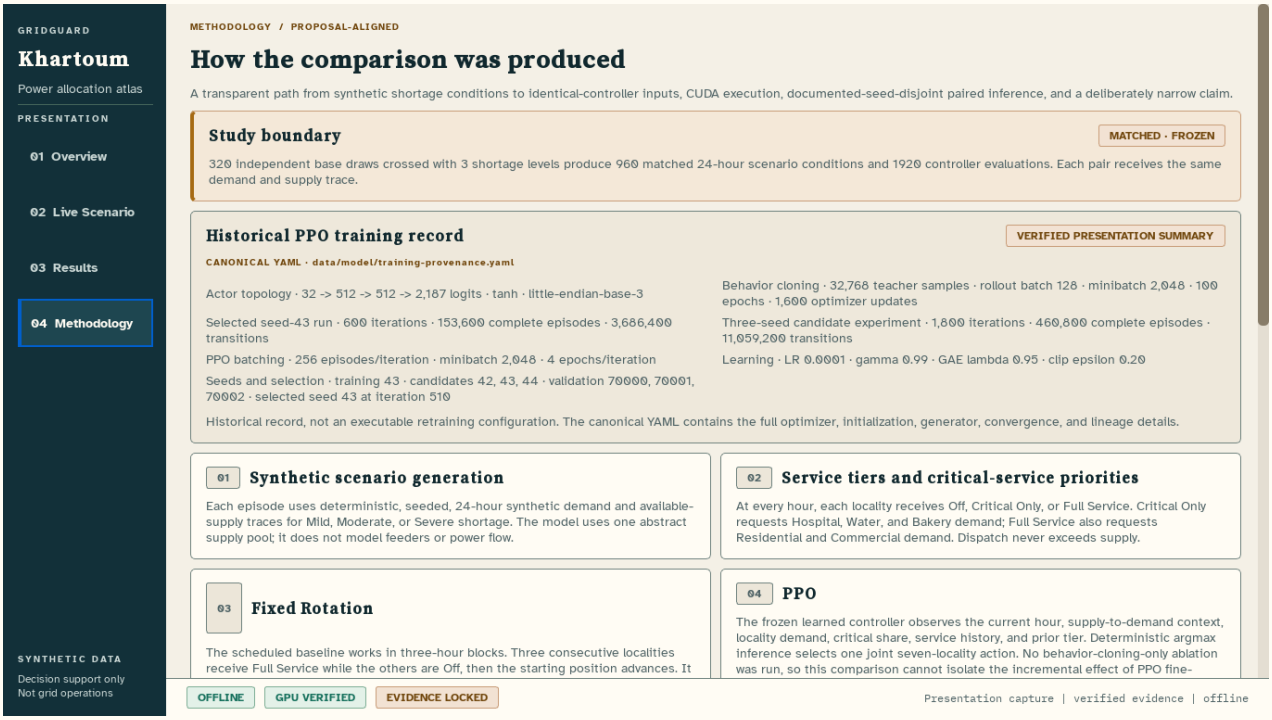
Overview - native offline desktop view captured on the presentation laptop.



Live scenario - native offline desktop view captured on the presentation laptop.



Verified results - native offline desktop view captured on the presentation laptop.



Methodology - native offline desktop view captured on the presentation laptop.

Results

Result statement

The reviewer-facing result is the documented-seed-disjoint holdout on seeds 81001-81010. It contains 320 independent base draws crossed with Mild, Moderate, and Severe shortage-only conditions: 960 matched scenario conditions and 1,920 controller-scenario evaluations. Under this synthetic protocol, PPO has lower critical-outage locality-hours, higher Jain fairness, and higher utilization than Fixed Rotation. Both controllers record zero power-balance violations.

All three deterministic 95% seed-stratified paired cluster intervals lie in the direction favoring PPO. That direction is an observed result, not a release criterion: the artifact verifier is sign-neutral and would accept a well-formed favorable, adverse, or ambiguous result. The finding applies to one frozen PPO actor, one supply-blind Fixed Rotation baseline, and the stated synthetic generator. It is decision-support evidence, not a real-grid operational claim.

The seeds are disjoint only from every documented training, validation, and model-selection seed in `data/model/training-provenance.yaml`. The holdout was not preregistered or independently custodied and cannot rule out undocumented activity. No behavior-cloning-only ablation was run, so this comparison cannot isolate the incremental effect of PPO fine-tuning.

Results provenance and evidence roles

The project keeps historical parity, documented seed disjointness, and training lineage distinct.

Evidence role	File and SHA-256	Permitted interpretation
Accepted aggregate source	`data/evidence/frozen-benchmark.json` - `3d720ba87a747a5780119decd7144b4051fc36fc72ab50f229f5750ad749fd15`	Preserve the accepted 18 batch-cell summaries; it contains no historical episode rows or intervals
Accepted reproduction evidence	`data/evidence/paired-benchmark.json` - `7f2009c4f7efbbedf098069ced9218addec51d55a18b969e6c54433027020af5`	Reproduce 96 independent base draws, 288 matched scenario conditions, 576 controller evaluations, paired rows, and frozen parity; seeds `50001-50003` participated in model selection, so this is not an independently custodied final test
Documented-seed-disjoint holdout	`data/evidence/robustness-supplement.json` - `fabb5a2f13ab31cd51f641f4e6a6db3cce8a28a6e2c4fa144612f634bad0beb1`	Report the 320-draw primary comparison, 24 sensitivity variants, safety invariant, and synthetic forecast evaluation; disjointness covers documented activity only and does not imply preregistration or independent custody
Historical PPO training record	`data/model/training-provenance.yaml` - `d2568bce26e9221afa8c266113a704f6fbf2370d3bc949f5aa6c72a7411ba1a8`	Distinguish the selected seed-43 run from the three-seed candidate experiment; scope hardware/convergence to the selected run; expose promotion checks, provenance categories, unavailable historical facts, and source lineage; this is not a bitwise-retraining bundle and historical gates are not independent headline evidence

`validate_paired_benchmark_payload` reconstructs closed schema, pairing, physical bounds, and arithmetic. `assert_paired_benchmark_integrity` adds locked provenance and direct frozen-cell parity. The robustness verifier independently reconstructs protocol counts, paired means, difference-in-differences,

forecast sufficient-statistic bounds, and source bindings. These are sign-neutral integrity paths; no favorable-result release gate exists.

Full numeric precision remains in canonical JSON. Human-facing prose, tables, UI, and slides use one decimal place for MWh and locality-hours, two decimals for percentages, and three decimals for the Training-objective score (diagnostic).

The display geography has independent provenance and no evidentiary role. OCHA/HDX administrative boundaries are pinned at SHA-256 `df1afbb337047402feb18d2dd26273c671c36761970d0c48c591c62f3ae3d496`; Natural Earth Nile linework is pinned at `0486a0f7318b5d7c576c13093b27f3f7aa6410329a7c54fb66eb37d3436cdafa`. Both are presentation context only and not electrical-network topology. The map does not enter scenario generation, controller input, dispatch, metrics, or evidence.

Documented-seed-disjoint primary outcomes

Every difference is PPO - Fixed Rotation. Percentage-point differences are shown for ratio outcomes. Values are rounded only for presentation.

Primary outcome	Fixed Rotation	PPO	Mean paired difference	Deterministic 95% interval	Direction
Critical-outage locality-hours	102.1	20.2	-81.9	[-82.5, -81.2]	Favorable
Jain fairness	87.76%	96.91%	+9.15 percentage points	[+9.11, +9.19]	Favorable
Utilization	71.21%	97.61%	+26.39 percentage points	[+26.34, +26.45]	Favorable

Critical-outage locality-hours can range from 0 to 168 per episode. It counts each locality-hour in which critical-sector service is below the explicit 99% threshold. Jain fairness is the mean of hourly fairness over cumulative locality service ratios. Utilization is the mean hourly served power divided by `min(available supply, demand)`. These definitions are not interchangeable with the reward components or with feeder-level outage and stability measures.

The safety result is separate from the primary table. Maximum power-balance violation rate is 0.00% across the base comparison and all 24 sensitivity variants for both controllers under the 0.001 MW simulator tolerance. This verifies an implemented conservation invariant; it is not a claim about voltage, frequency, protection, or real-grid safety.

Accepted reproduction context

The accepted reproduction remains useful because it exactly preserves the prior committee-facing comparison while exposing its evidence role. It has 96 independent base draws, 288 matched scenario conditions, and 576 controller-scenario evaluations. Its legacy numeric projection is unchanged after reward reframing.

Outcome	Fixed Rotation	PPO	Public role
Training-objective score (diagnostic)	13.969	23.512	Diagnostic; shares components with reported outcomes
Critical-service ratio	42.63%	99.72%	Supporting
Critical-outage locality-hours	102.1	18.7	Primary
Jain fairness	87.76%	96.92%	Primary
Utilization	71.22%	97.65%	Primary
Served energy	4,551.7 MWh	6,560.2 MWh	Supporting
Curtailed energy	6,142.4 MWh	4,133.9 MWh	Complementary energy identity

Outcome	Fixed Rotation	PPO	Public role
Switching rate	25.00%	21.89%	Secondary
Power-balance violation rate	0.00%	0.00%	Safety

The accepted critical-outage paired difference is -83.4 locality-hours with a deterministic 95% interval of [-84.6, -82.2]. The Training-objective score difference is +9.543, but it is not an independent primary outcome: the objective directly includes critical service, utilization, fairness, switching, and oversubscription. Served and curtailed energy are likewise complementary under the episode energy identity and are not two independent endpoints.

The accepted cell summaries still match the frozen aggregate artifact within every locked tolerance. That parity proves source-compatible reproduction; it does not make the three reused seeds independently custodied.

Sensitivity across illustrative demand allocation

The one-factor-at-a-time analysis changes each of seven locality weights and five sector shares by -25% and +25%, renormalizing the affected vector. The 24 variants plus the unchanged base give 25 parameterizations. Each variant reuses the same 320 base draws across all three shortage levels and both controllers while holding the base available-supply trace exactly fixed, producing 24,000 matched sensitivity scenarios and 48,000 controller-scenario evaluations when the base is included.

All 144 sensitivity intervals are descriptive, pointwise 95% intervals and are unadjusted for multiple comparisons. They do not establish family-wide statistical significance.

For every one of the 24 variants, the paired interval remains favorable for PPO on all three primary outcomes. Difference-in-differences asks a different question: whether the perturbed paired effect is larger or smaller than the base paired effect. Those classifications are mixed:

Difference-in-differences outcome	Favorable	Ambiguous	Adverse
Critical-outage locality-hours	4	1	19
Jain fairness	6	9	9
Utilization	6	5	13

Thus, the direction of the controller comparison persists in this local +/-25% neighborhood, but many perturbations reduce the size of the base advantage. The study supports bounded local sensitivity, not parameter invariance, interaction coverage, global robustness, or calibration to Khartoum.

Synthetic forecast comparison

Both display methods are evaluated on the same 320 documented-seed-disjoint synthetic demand traces and common target hours 3-23. The three-hour trailing mean uses only hours $t-3$ through $t-1$; the one-hour persistence baseline uses $t-1$. Each method has 235,200 locality-sector predictions and 6,720 system-total predictions. Errors are in MW at each one-hour target; locality-sector statistics pool episode x target x 7 x 5 absolute cell errors, while system-total error is calculated after summing all 35 locality-sector residuals at a target. Squared-error sums use MW squared, MAE uses MW, and RMSE uses MW.

Aggregation	Method	MAE	RMSE	Lower-error method
Locality-sector MW	Three-hour trailing mean	2.338	3.745	Persistence
Locality-sector MW	One-hour persistence	1.346	2.178	Persistence
System-total MW	Three-hour trailing mean	42.725	49.973	Persistence

Aggregation	Method	MAE	RMSE	Lower-error method
System-total MW	One-hour persistence	24.377	28.584	Persistence

Persistence has lower error on all four stored measures. Trailing-minus-persistence differences are +0.992 MW locality-sector MAE, +1.567 MW locality-sector RMSE, +18.348 MW system MAE, and +21.390 MW system RMSE. This is a useful negative result: the more complex display projection does not beat the simple baseline under this synthetic generator. It remains display-only and is never PPO input. No real-world forecast-accuracy claim follows.

Baseline interpretation

Fixed Rotation serves three consecutive localities at Full Service for each three-hour block and turns the other four Off. It does not observe supply, demand, critical need, or accumulated fairness; it cannot select Critical Only. In the accepted reproduction its critical-service ratios are 43.27%, 43.27%, and 41.35%, while utilization rises from 54.26% to 68.68% to 90.73% across Mild, Moderate, and Severe shortages. The utilization rise is a denominator effect as usable supply falls, not evidence that the schedule adapts.

Mean unused available energy is 2,158.7 MWh for Fixed Rotation versus 150.2 MWh for PPO. Part of the utilization difference is therefore attributable to the supply-blind baseline design, which uses only Off and Full Service. The evidence supports comparison with this proposal baseline only. It does not establish PPO superiority over a supply-aware critical-preserving heuristic or every rule-based controller.

Interpretation against the proposal questions

- Critical services: PPO reduces the modeled critical-outage burden relative to Fixed Rotation, but proportional dispatch does not implement feeder-level within-hour hospital priority.
- Fairness: PPO raises mean hourly Jain fairness of cumulative locality service ratios in the locked synthetic generator.
- Distribution efficiency: PPO raises utilization; supporting served energy and complementary curtailment explain the energy allocation without being counted as extra primary outcomes.
- Prediction: one-hour persistence beats the display-only trailing mean on all stored synthetic MAE/RMSE measures.
- Data and rules needed for realism (RQ3): a field-facing model would need feeder/locality demand, independent supply and generation constraints, network topology, critical-facility loads and priorities, outage/restoration histories, operating and switching rules, weather/calendar effects, and validated asset-to-locality mappings.
- Optimization: Adam optimized PPO parameters during historical training; CUDA accelerates computation. The desktop implements no separate online mathematical dispatch optimizer.

Limitations of the result

- The scenarios are synthetic, Khartoum-inspired, and uncalibrated. Supply is constructed partly from synthetic demand, so matched scenarios do not remove shared generator misspecification.
- Fixed Rotation is supply-blind and limited to Off and Full Service. A stronger supply-aware baseline is outside the accepted scope but necessary future work.
- Performance relative to the global optimum is unknown; no valid upper-bound comparator is included.
- When requests exceed supply, one proportional factor scales critical and non-critical sectors together. Critical-service outcomes are not feeder-level restoration evidence.

- The actor enumerates 3^N actions (3^7 = 2,187 here), limiting scalability beyond seven localities.
- The operating domain is shortage-only: Mild, Moderate, and Severe. Surplus, normal operation, recovery, and restoration transients are not evaluated.
- Bootstrap intervals are conditional on the fixed synthetic seed strata and shortage design. Ten documented-seed-disjoint seeds improve separation and coverage but were not preregistered or independently custodied and do not represent all future conditions.
- Sensitivity intervals are pointwise and unadjusted for multiple comparisons; no family-wide significance claim is supported.
- No behavior-cloning-only ablation was run, so the incremental contribution of PPO fine-tuning beyond the initialization is not isolated.
- The +/-25% one-factor sensitivity excludes interactions and cannot replace calibration data.
- Forecast errors use synthetic traces only. Persistence's lower error does not establish a real-world forecasting conclusion.
- Power-balance compliance is a simulator invariant, not evidence of real-grid stability or operational safety.
- Administrative boundaries and Nile linework are presentation context only; they contribute no scientific input.

Discussion

The documented-seed-disjoint holdout is the reviewer-facing result because its ten seeds are disjoint from every documented training, validation, and model-selection seed in `data/model/training-provenance.yaml`. This does not establish preregistration or independent custody and cannot rule out undocumented activity. The accepted 96-draw artifact remains valuable for frozen parity, but it is labeled accepted reproduction evidence because its seeds participated in model selection. Both layers use matched controller pairs and sign-neutral verification; no favorable interval is a release condition.

The three primary outcomes avoid double-counting the optimized reward: critical-outage locality-hours captures outage burden, Jain fairness captures distribution between localities, and utilization captures use of available energy. The reward is a Training-objective score (diagnostic), critical-service ratio and served energy are supporting, curtailed energy is complementary, and power-balance violation is separate safety. This framing does not erase the accepted numbers; it limits what each number is allowed to prove.

Fixed Rotation is transparent but supply-blind, is limited to Off and Full Service, and cannot select Critical Only. Its rising utilization as shortage worsens is a denominator effect, not adaptive behavior. Mean unused supply in the accepted reproduction is 2,158.7 MWh for Fixed Rotation versus 150.2 MWh for PPO, so part of the utilization contrast reflects baseline design. A supply-aware critical-preserving heuristic is necessary future work before making broad claims against rule-based control. Performance relative to the global optimum is unknown.

These findings remain about allocation logic in this simulator. The single global supply pool omits feeder capacity, voltage, frequency, protection, losses, restoration procedures, operator constraints, and sub-hour dynamics. When requests exceed supply, proportional dispatch scales critical and non-critical demand together. The demand profiles, shortage construction, reward weights, and outage threshold are illustrative assumptions rather than utility measurements. Matched scenarios isolate controllers but cannot repair a misspecified generator.

The sensitivity study perturbs 12 allocation components in both directions, giving 24 one-factor-at-a-time variants and 25 parameterizations including the base. It measures local dependence on those assumptions, not all plausible parameter interactions. Its intervals are descriptive, pointwise 95%, and unadjusted for multiple comparisons; no family-wide significance claim follows. Forecast MAE and RMSE compare the three-hour trailing mean with one-hour persistence on synthetic targets; they do not establish real-world forecast accuracy. The evaluated operating domain is shortage-only and says nothing about surplus, recovery, normal operation, or restoration transients. The PPO actor also enumerates 3^N joint actions (3^7 = 2,187 here), which limits scaling beyond seven localities.

The presentation design reinforces that boundary. The real administrative map is more legible and recognizable than the former abstract drawing, but visual realism must not be mistaken for model realism. OCHA/HDX boundaries and Natural Earth rivers are presentation context only and are not electrical-network topology. Likewise, the live aggregate table helps the committee compare the two controllers on one selected run, while the Results page remains bound to the separately verified frozen evidence.

Conclusion

GridGuard Khartoum fulfills the accepted proposal as a focused, auditable decision-support simulation and academic presentation package. It demonstrates a complete chain from deterministic synthetic scenario generation, through matched PPO and Fixed Rotation execution, to canonical primary and safety metrics, paired uncertainty, sensitivity, forecast comparison, provenance checks, and an offline native interface. The conclusion is intentionally bounded: it establishes comparative behavior for one frozen actor and one weak baseline in the reviewed simulator, not operational superiority on a real grid, over all rule-based methods, or relative to an optimum.

The release also improves how that conclusion can be inspected. Sign-neutral evidence validation tests schema, arithmetic, physical identities, pairing, source bindings, and frozen parity without assuming a favorable result. The strict training YAML makes each recovered parameter, provenance category, scope, historical unknown, and selection caveat discoverable. The live view exposes matched episode metrics without altering the benchmark, and the source-pinned administrative map gives geographic orientation without entering the scientific pipeline. These separations make the project easier to explain and harder to misrepresent.

The training record also makes the attribution boundary explicit. No behavior-cloning-only ablation was run, so this comparison cannot isolate the incremental effect of PPO fine-tuning.

Future work should begin with representative operational data and an engineering network model, followed by independent validation of constraints, reward choices, uncertainty, robustness, and human operating procedures. Only after those steps could field-facing trials be considered. Until then, GridGuard Khartoum remains what its evidence supports: a reproducible software demonstration of how PPO can assist simplified load-shedding decisions under synthetic shortage conditions.

Accepted reproduction SHA-256: 7f2009c4f7efbbedf098069ced9218addec51d55a18b969e6c54433027020af5
Documented-seed-disjoint holdout SHA-256: fabb5a2f13ab31cd51f641f4e6a6db3cce8a28a6e2c4fa144612f634bad0beb1
Training record: data/model/training-provenance.yaml

References

1 GridGuard Khartoum project team. Graduation Project Proposal: AI-Based Smart Load Shedding and Energy Management System for Developing Countries. Unpublished project proposal, archived at `docs/source/graduation-project-proposal.pdf`.

2 Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. "Proximal Policy Optimization Algorithms." arXiv preprint arXiv:1707.06347, 2017. DOI: 10.48550/arXiv.1707.06347.

3 Jain, R. K., Chiu, D.-M. W., and Hawe, W. R. "A Quantitative Measure of Fairness and Discrimination for Resource Allocation in Shared Computer Systems." DEC Research Report TR-301, 1984.

4 Efron, B., and Tibshirani, R. J. An Introduction to the Bootstrap. Chapman and Hall/CRC, 1993. DOI: 10.1201/9780429246593.

5 Paszke, A., Gross, S., Massa, F., et al. "PyTorch: An Imperative Style, High-Performance Deep Learning Library." Advances in Neural Information Processing Systems 32, 2019.

6 Okuta, R., Unno, Y., Nishino, D., Hido, S., and Loomis, C. "CuPy: A NumPy-Compatible Library for NVIDIA GPU Calculations." Proceedings of the Workshop on Machine Learning Systems at NIPS 2017, 2017.

7 United Nations Office for the Coordination of Humanitarian Affairs. Sudan Administrative Boundaries (COD-AB), revision v03. Humanitarian Data Exchange, valid from 31 August 2020. CC BY 3.0 IGO. Archived source bytes and SHA-256 pins are recorded in `src/gridguard/assets/map/THIRD_PARTY_DATA.md`.

8 Natural Earth. 1:10m Rivers and Lake Centerlines, version 5.0.0. Public-domain geographic data. The immutable source revision, selected features, transformation, and SHA-256 pin are recorded in `src/gridguard/assets/map/THIRD_PARTY_DATA.md`.

9 World Bank. Accelerating Sustainable and Clean Energy and Digital Access Transformation in Sudan (P507788). Project Information Document, Appraisal Stage, Report PIDIA01286, 1 May 2025. Official project document.

10 Heggie, A., Eager, D., McKinnon, K., and van der Weijde, A. H. "Power Rationing in a Long-Term Power Shortage." Energy Policy 121, 2018, pp. 202 - 210. DOI: 10.1016/j.enpol.2018.06.025.

11 Oluwasuji, O. I., Malik, O., Zhang, J., and Ramchurn, S. D. "Solving the Fair Electric Load Shedding Problem in Developing Countries." Autonomous Agents and Multi-Agent Systems 34, article 12, 2020. DOI: 10.1007/s10458-019-09428-8.

12 Poudel, S., and Dubey, A. "Critical Load Restoration Using Distributed Energy Resources for Resilient Power Distribution System." IEEE Transactions on Power Systems 34(1), 2019, pp. 52 - 63. DOI: 10.1109/TPWRS.2018.2860256.

13 Vázquez-Canteli, J. R., and Nagy, Z. "Reinforcement Learning for Demand Response: A Review of Algorithms and Modeling Techniques." Applied Energy 235, 2019, pp. 1072 - 1089. DOI: 10.1016/j.apenergy.2018.11.002.

14 Glavic, M. "(Deep) Reinforcement Learning for Electric Power System Control and Related Problems: A Short Review and Perspectives." Annual Reviews in Control 48, 2019, pp. 22 - 35. DOI: 10.1016/j.arcontrol.2019.09.008.

15 Nti, I. K., Teimeh, M., Nyarko-Boateng, O., and Adekoya, A. F. "Electricity Load Forecasting: A Systematic Review." Journal of Electrical Systems and Information Technology 7, article 13, 2020. DOI: 10.1186/s43067-020-00021-8.